

FEDERATED SCIENTIFIC INTELLIGENCE

Validation & Regulatory Readiness Platform

Anonymized Principal AI Architect Case Study

ARCHITECTURAL MANDATE	DESIGN THESIS
Establish a governed scientific intelligence platform that connects heterogeneous biomedical evidence to traceable retrieval, AI-assisted reasoning and repeatable regulatory workflows.	Probabilistic AI remains inside deterministic evidence, workflow, governance and audit boundaries. AI assists reasoning; it does not become scientific or regulatory authority.
PRODUCTION POSTURE	ARCHITECTURE EVIDENCE
Federated data • GraphRAG • distributed scientific processing • controlled inference • provenance • guardrails • evaluation • SRE • IaC	3×+ evidence-processing throughput • 40%+ manual research reduction • 30%+ regulatory workflow-cycle reduction • sub-second evidence retrieval • 95%+ provenance coverage • 70%+ repeatable workflow automation

Architecture is treated as a system of explicit boundaries: data authority, evidence lineage, knowledge, context, inference, workflow, trust, operations and auditability.

01 • EXECUTIVE ARCHITECTURE OVERVIEW

The platform is designed as a governed scientific intelligence substrate rather than an LLM feature. The key architectural separation is between authoritative data/evidence, knowledge and retrieval, probabilistic inference, deterministic workflow control, and production governance.

Architecture layer	Primary responsibility	Hard architectural question
Data / Evidence	Authoritative scientific sources, evidence objects, lineage	What is authoritative and how is it traceable?
Knowledge	Neo4j relationships + semantic retrieval	When is graph structure required beyond similarity search?
Context	Evidence admissibility and bounded context	What evidence is allowed into model context?
Inference	Amazon Bedrock + NVIDIA Triton boundary	Which serving path satisfies workload control requirements?
Workflow	Scientific / regulatory orchestration	Where must execution remain deterministic?
Trust	Guardrails, security, governance	Where can AI behavior be constrained before side effects?
Operations	OTel, CloudWatch, SLI/SLO, evaluation	How is behavior measurable and recoverable?

Principal-architect invariant: the model is a component inside the architecture; source evidence, policy, workflow state and production controls remain outside the model's authority.

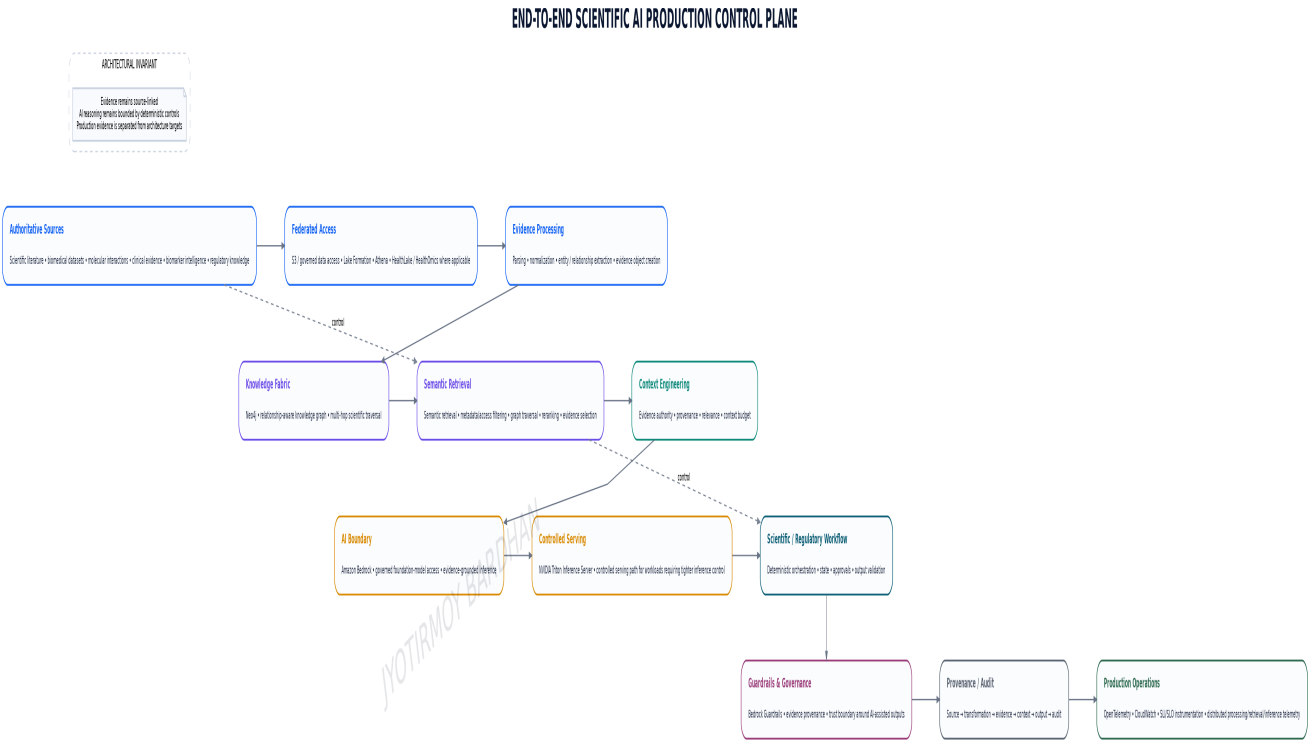
02 • ARCHITECTURE EVIDENCE MAP

#	Architecture evidence	Question answered
1	END-TO-END SCIENTIFIC AI PRODUCTION CONTROL PLANE	How does end-to-end scientific ai production control plane become an explicit production boundary?
2	ARCHITECTURE EVIDENCE MAP	How does architecture evidence map become an explicit production boundary?
3	AMBIGUITY → ARCHITECTURE	How does ambiguity → architecture become an explicit production boundary?
4	REQUIREMENTS & NFR ARCHITECTURE	How does requirements & nfr architecture become an explicit production boundary?
5	CONSTRAINTS → ARCHITECTURAL RESOLUTION	How does constraints → architectural resolution become an explicit production boundary?
6	ARCHITECTURE DECISION FLOW	How does architecture decision flow become an explicit production boundary?
7	TARGET-STATE HLD	How does target-state hld become an explicit production boundary?
8	CONTROL PLANE vs DATA PLANE	How does control plane vs data plane become an explicit production boundary?
9	COMPONENT BOUNDARIES & OWNERSHIP	How does component boundaries & ownership become an explicit production boundary?
10	SCIENTIFIC DATA INGESTION & PROCESSING	How does scientific data ingestion & processing become an explicit production boundary?
11	EVIDENCE PROVENANCE & LINEAGE	How does evidence provenance & lineage become an explicit production boundary?
12	GRAPHRAG + SEMANTIC RETRIEVAL KNOWLEDGE ARCHITECTURE	How does graphrag + semantic retrieval knowledge architecture become an explicit production boundary?
13	RETRIEVAL LATENCY DECOMPOSITION	How does retrieval latency decomposition become an explicit production boundary?
14	CONTEXT ENGINEERING & EVIDENCE BUDGET	How does context engineering & evidence budget become an explicit production boundary?
15	MODEL GATEWAY & INFERENCE ROUTING	How does model gateway & inference routing become an explicit production boundary?
16	MANAGED BEDROCK vs CONTROLLED TRITON DECISION	How does managed bedrock vs controlled triton decision become an explicit production boundary?
17	DISTRIBUTED SCIENTIFIC PROCESSING	How does distributed scientific processing become an explicit production boundary?
18	EVENT-DRIVEN ARCHITECTURE	How does event-driven architecture become an explicit production boundary?
19	SCIENTIFIC / REGULATORY WORKFLOW ORCHESTRATION	How does scientific / regulatory workflow orchestration become an explicit production boundary?
20	REGULATORY READINESS CONTROL PLANE	How does regulatory readiness control plane become an explicit production boundary?

The diagram package expands the architecture into processing, retrieval, inference, security, SRE, evaluation, auditability, deployment, traceability and operating-model evidence.

01 • END-TO-END SCIENTIFIC AI PRODUCTION CONTROL PLANE

How does end-to-end scientific ai production control plane behave as an explicit architecture boundary rather than a generic technology stack?



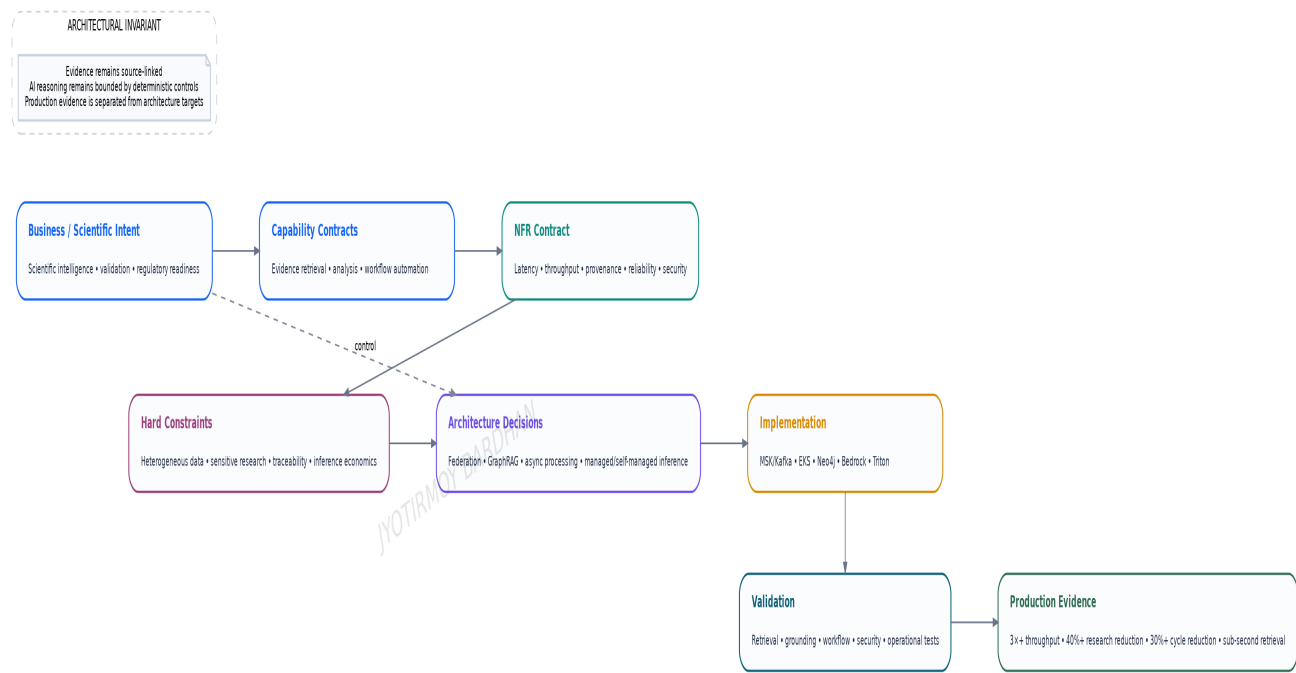
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

The end-to-end path deliberately moves from authoritative evidence to bounded context before inference. Provenance, guardrails, workflow validation and operational telemetry remain independent control surfaces.

02 • ARCHITECTURE EVIDENCE MAP

How does architecture evidence map behave as an explicit architecture boundary rather than a generic technology stack?

ARCHITECTURE EVIDENCE MAP



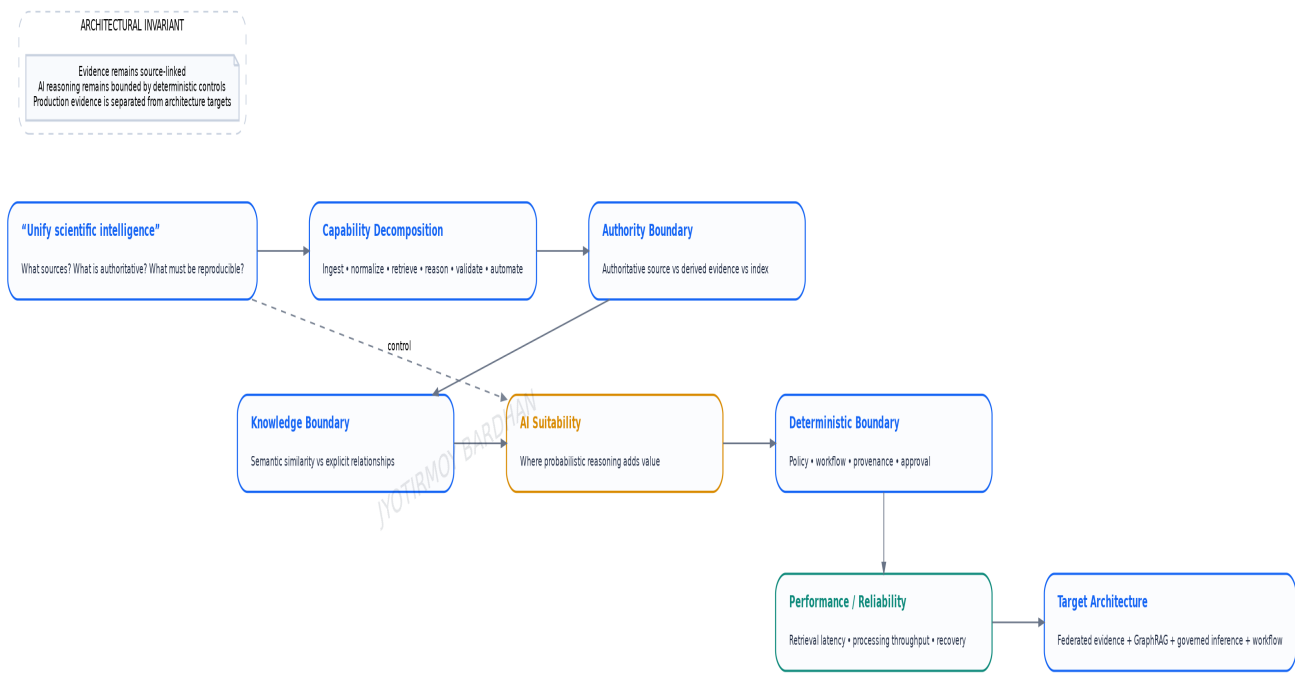
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

The evidence map converts architecture into proof obligations: each major decision has a requirement, constraint, implementation mechanism and measurable production consequence.

03 • AMBIGUITY → ARCHITECTURE

How does ambiguity → architecture behave as an explicit architecture boundary rather than a generic technology stack?

AMBIGUITY → ARCHITECTURE



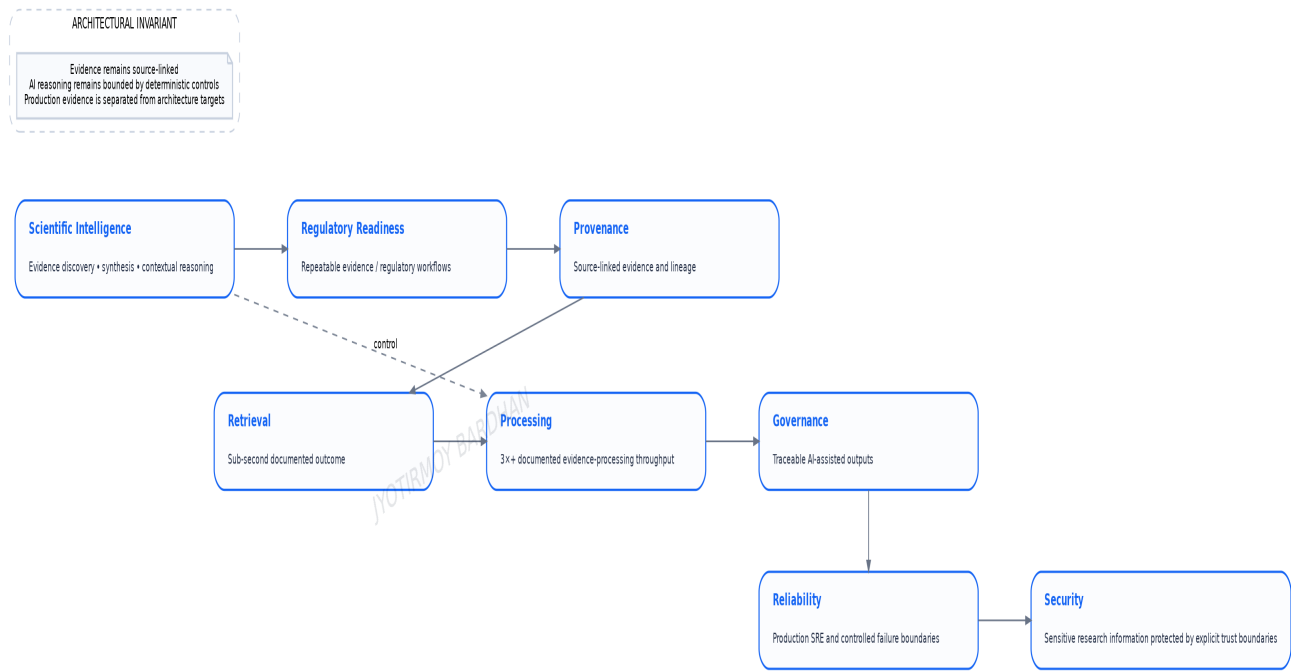
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Ambiguous scientific requirements are interrogated across authority, evidence, relationship structure, AI suitability, deterministic controls and NFRs before technology selection.

04 • REQUIREMENTS & NFR ARCHITECTURE

How does requirements & nfr architecture behave as an explicit architecture boundary rather than a generic technology stack?

REQUIREMENTS & NFR ARCHITECTURE



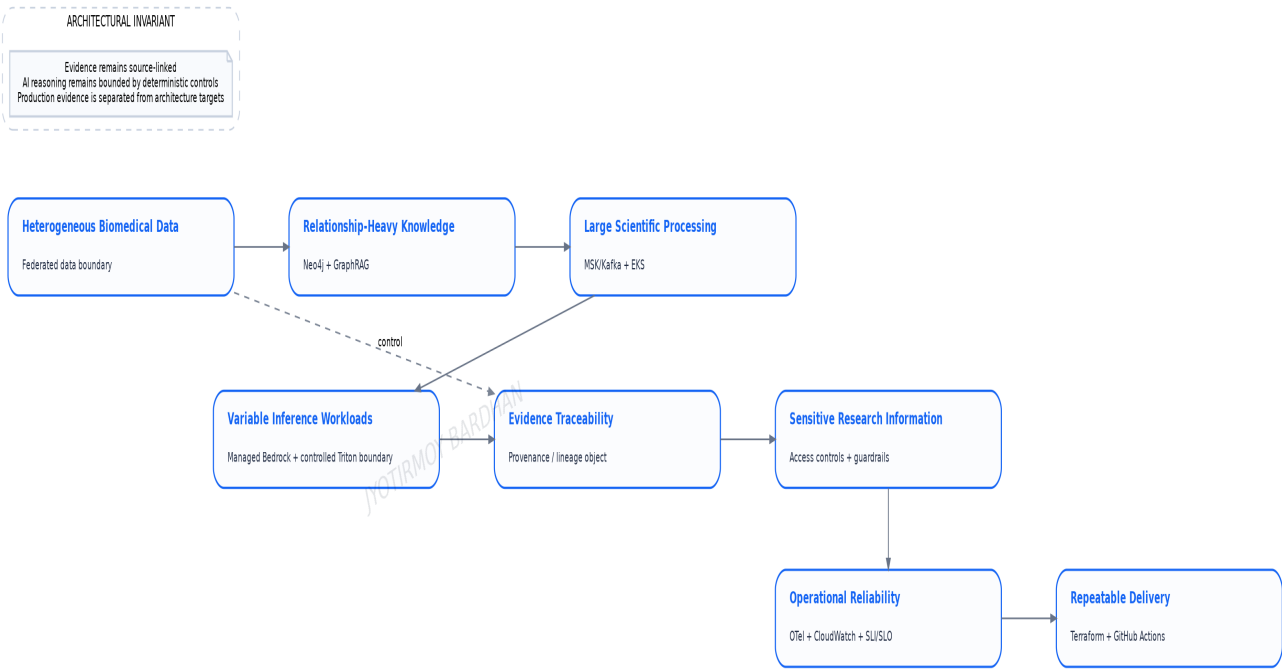
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

NFRs directly drive architecture: retrieval, processing, provenance, security and reliability requirements shape component boundaries and validation.

05 • CONSTRAINTS → ARCHITECTURAL RESOLUTION

How does constraints → architectural resolution behave as an explicit architecture boundary rather than a generic technology stack?

CONSTRAINTS → ARCHITECTURAL RESOLUTION



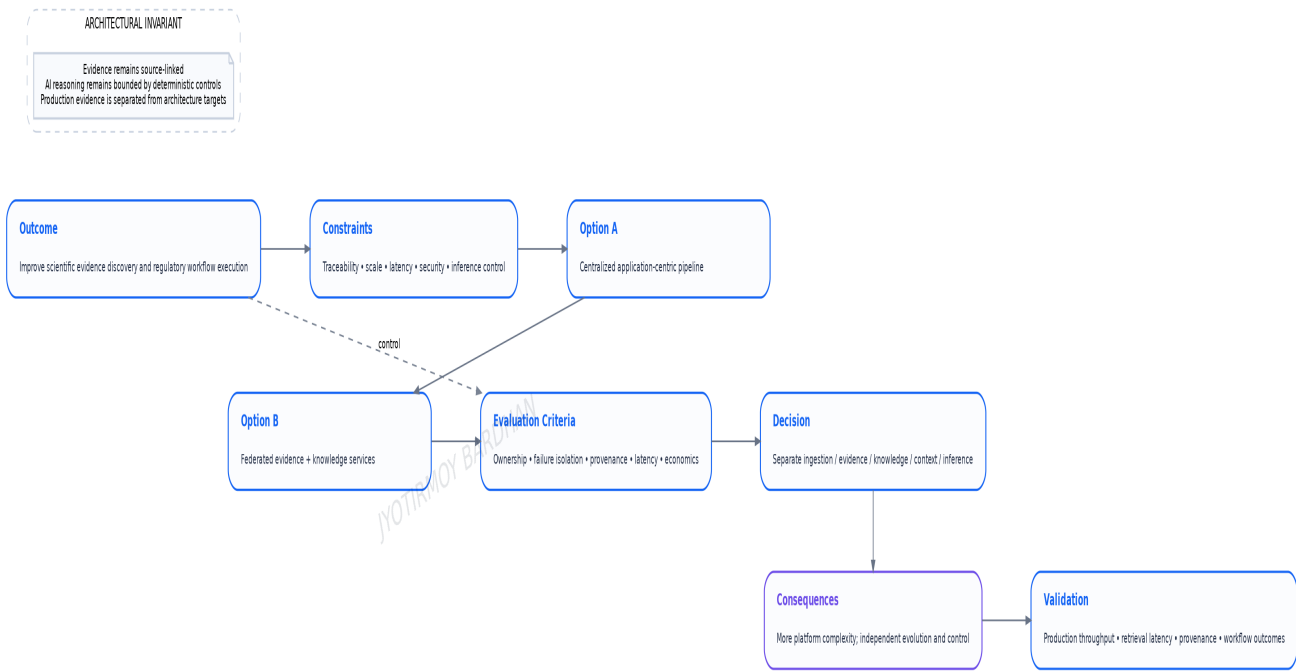
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

The platform resolves domain constraints through federation, graph-aware knowledge, asynchronous processing, controlled inference, provenance and operational controls.

06 • ARCHITECTURE DECISION FLOW

How does architecture decision flow behave as an explicit architecture boundary rather than a generic technology stack?

ARCHITECTURE DECISION FLOW

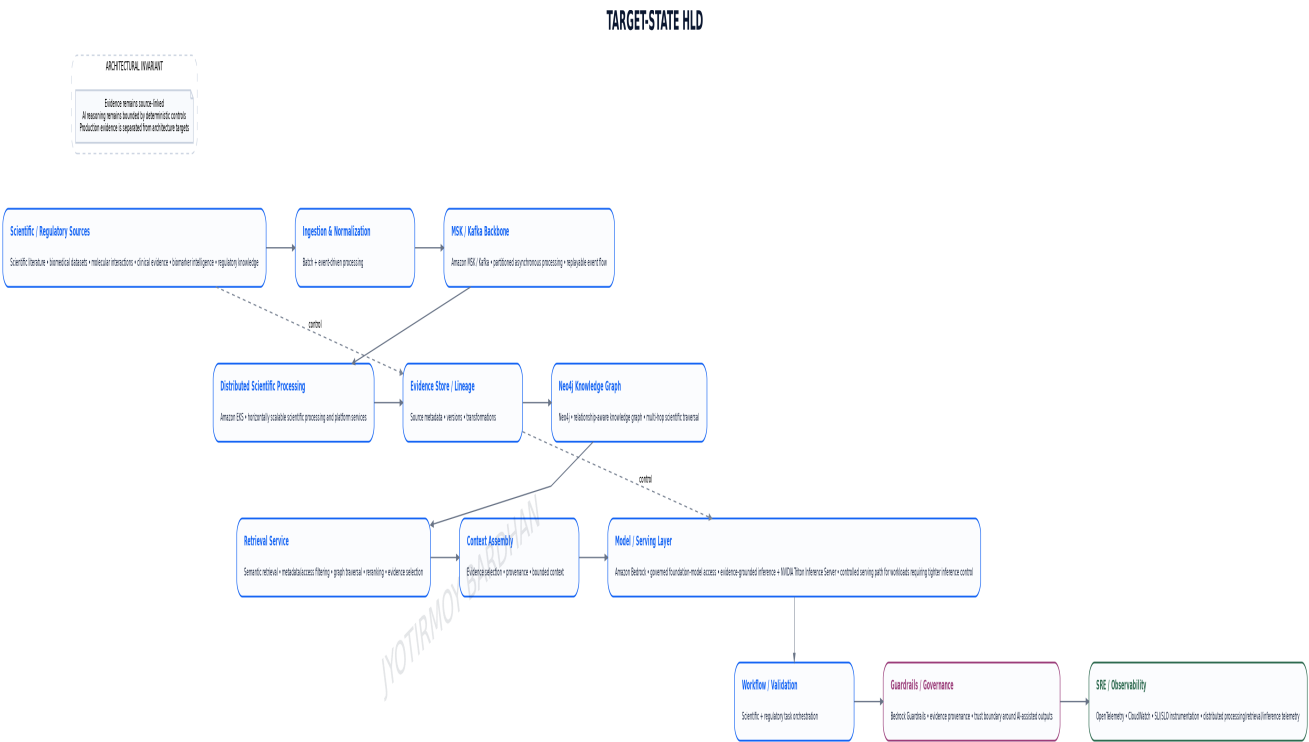


Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Architecture decisions are evaluated against explicit criteria and accepted with known trade-offs rather than selecting technologies by familiarity.

07 • TARGET-STATE HLD

How does target-state hld behave as an explicit architecture boundary rather than a generic technology stack?



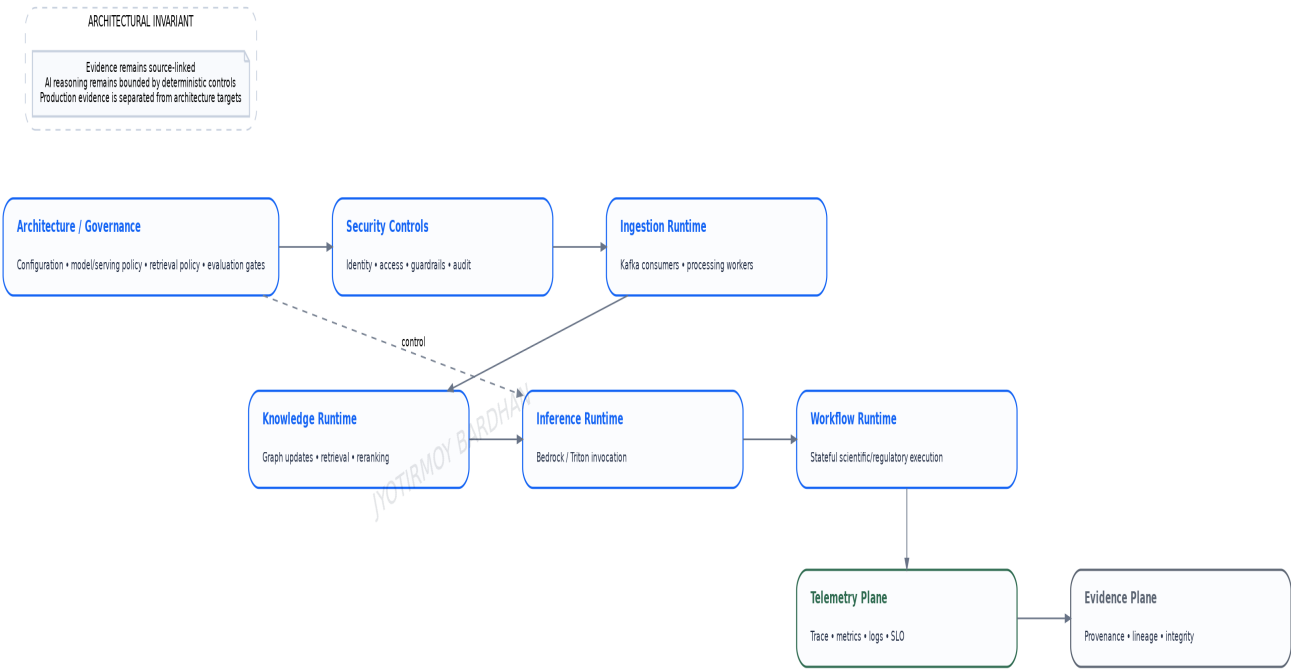
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

The HLD separates source data, event processing, evidence, knowledge, retrieval, context, inference, workflow, trust and operations so each can evolve independently.

08 • CONTROL PLANE vs DATA PLANE

How does control plane vs data plane behave as an explicit architecture boundary rather than a generic technology stack?

CONTROL PLANE vs DATA PLANE



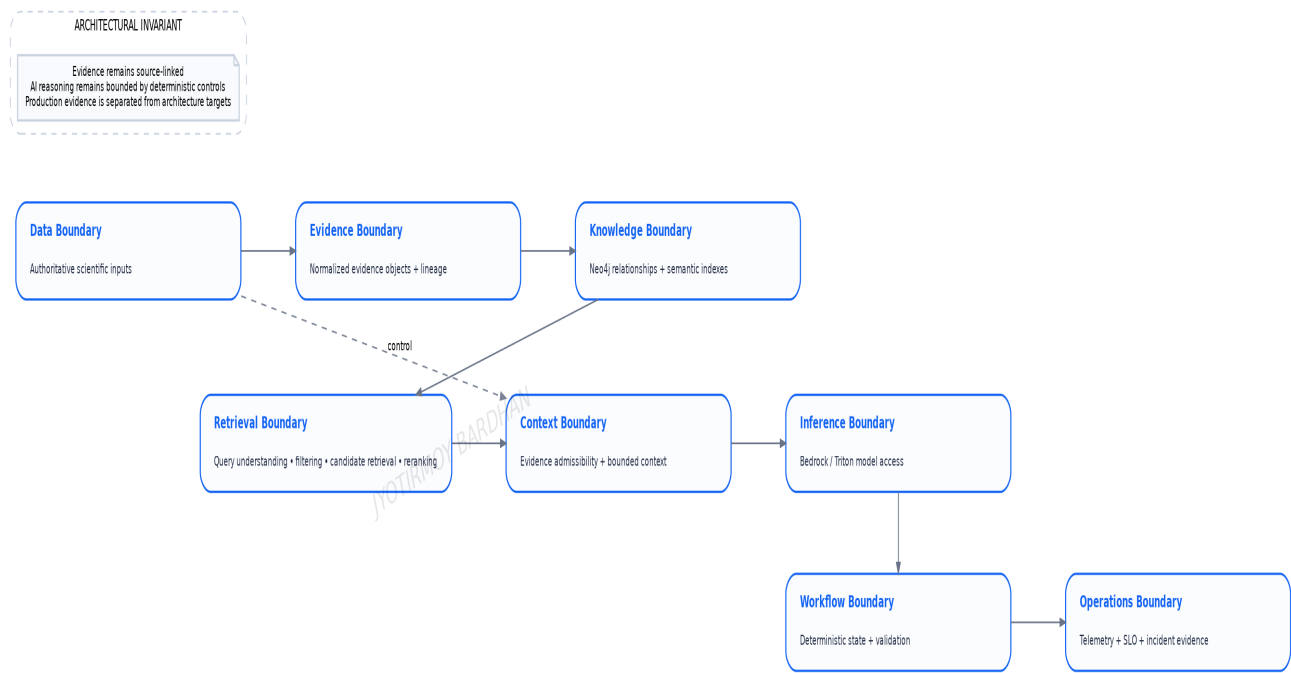
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Control-plane intent governs data-plane execution. This keeps governance and configuration from becoming coupled to long-running scientific processing.

09 • COMPONENT BOUNDARIES & OWNERSHIP

How does component boundaries & ownership behave as an explicit architecture boundary rather than a generic technology stack?

COMPONENT BOUNDARIES & OWNERSHIP



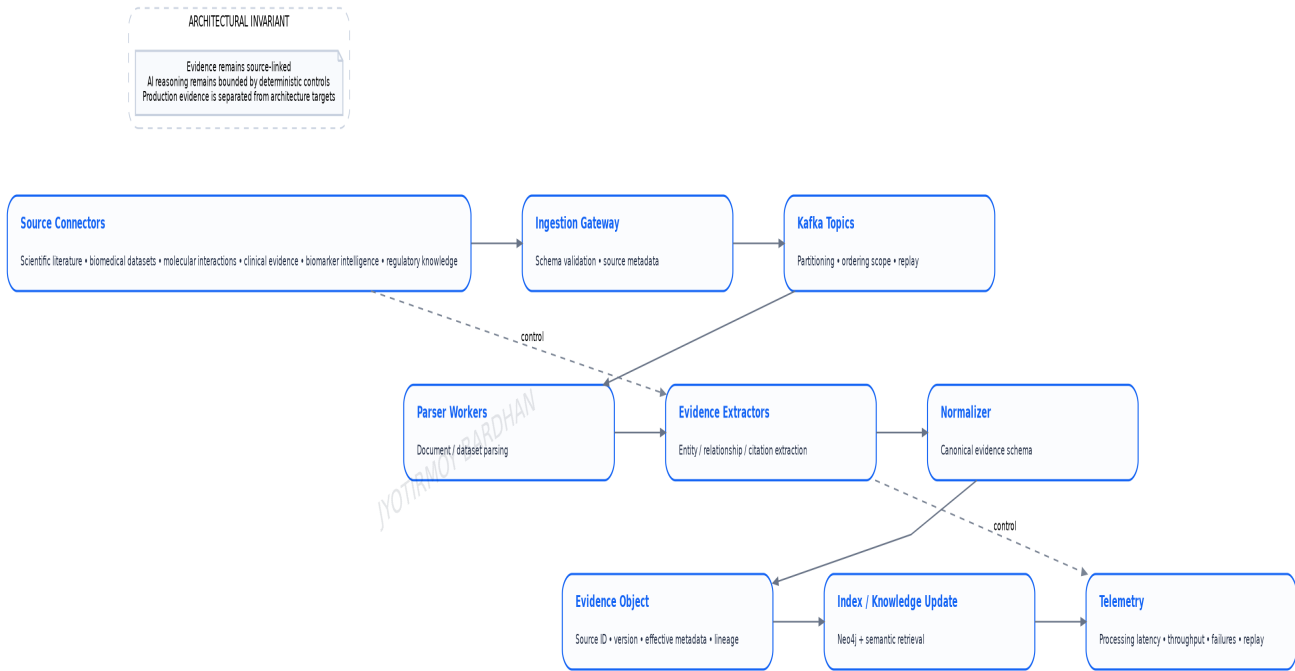
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Each boundary has a defined owner and a prohibition on taking ownership of another domain's authority.

10 • SCIENTIFIC DATA INGESTION & PROCESSING

How does scientific data ingestion & processing behave as an explicit architecture boundary rather than a generic technology stack?

SCIENTIFIC DATA INGESTION & PROCESSING



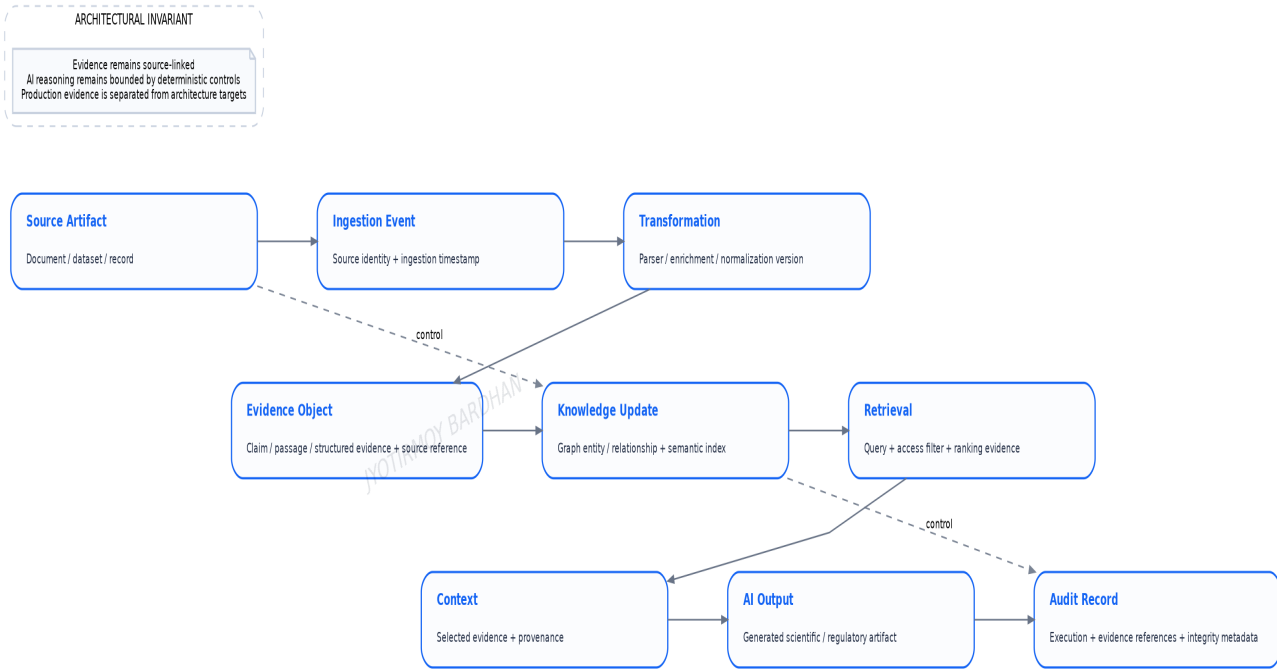
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

MSK/Kafka provides asynchronous decoupling while EKS provides scalable processing capacity; replay and failure boundaries are explicit.

11 • EVIDENCE PROVENANCE & LINEAGE

How does evidence provenance & lineage behave as an explicit architecture boundary rather than a generic technology stack?

EVIDENCE PROVENANCE & LINEAGE



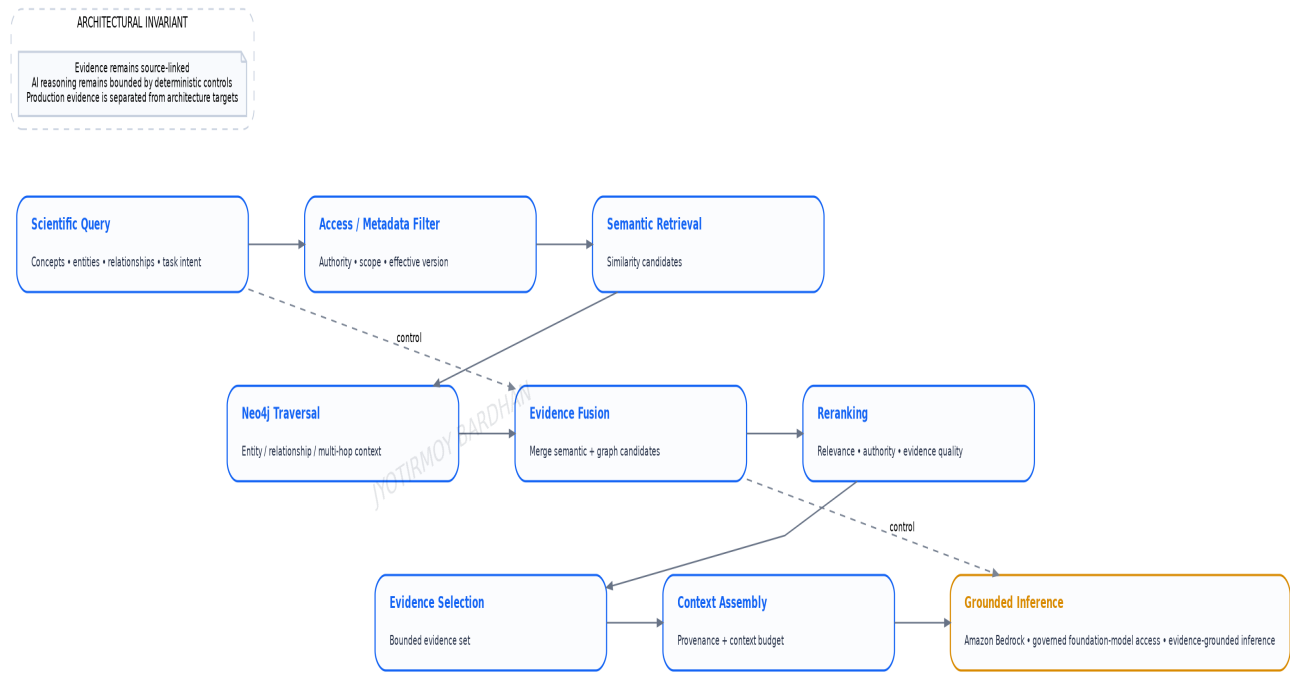
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Provenance follows evidence through transformation, knowledge, retrieval, context and output. The documented production outcome is 95%+ provenance coverage.

12 • GRAPHRAG + SEMANTIC RETRIEVAL KNOWLEDGE ARCHITECTURE

How does graphrag + semantic retrieval knowledge architecture behave as an explicit architecture boundary rather than a generic technology stack?

GRAPHRAG + SEMANTIC RETRIEVAL KNOWLEDGE ARCHITECTURE



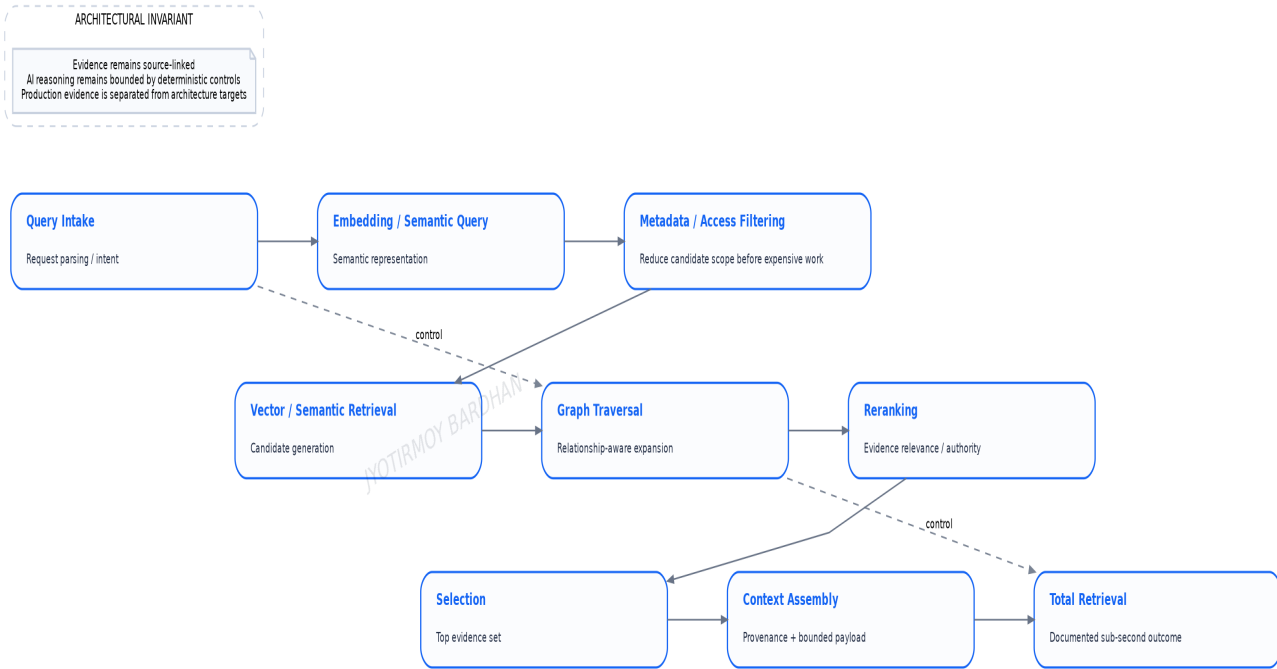
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

GraphRAG complements semantic retrieval: vector search finds semantic candidates while Neo4j supplies explicit relationships and multi-hop scientific context.

13 • RETRIEVAL LATENCY DECOMPOSITION

How does retrieval latency decomposition behave as an explicit architecture boundary rather than a generic technology stack?

RETRIEVAL LATENCY DECOMPOSITION



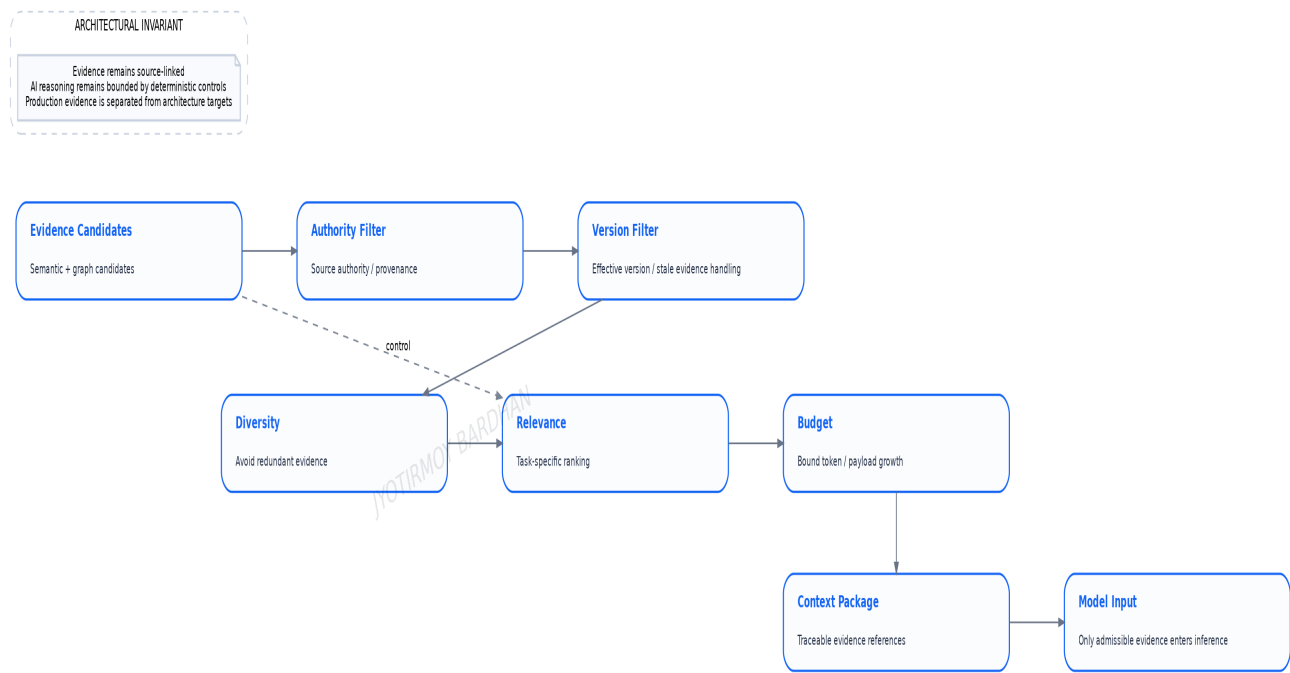
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Retrieval latency is decomposed by stage so a sub-second result can be explained rather than treated as a black-box benchmark.

14 • CONTEXT ENGINEERING & EVIDENCE BUDGET

How does context engineering & evidence budget behave as an explicit architecture boundary rather than a generic technology stack?

CONTEXT ENGINEERING & EVIDENCE BUDGET



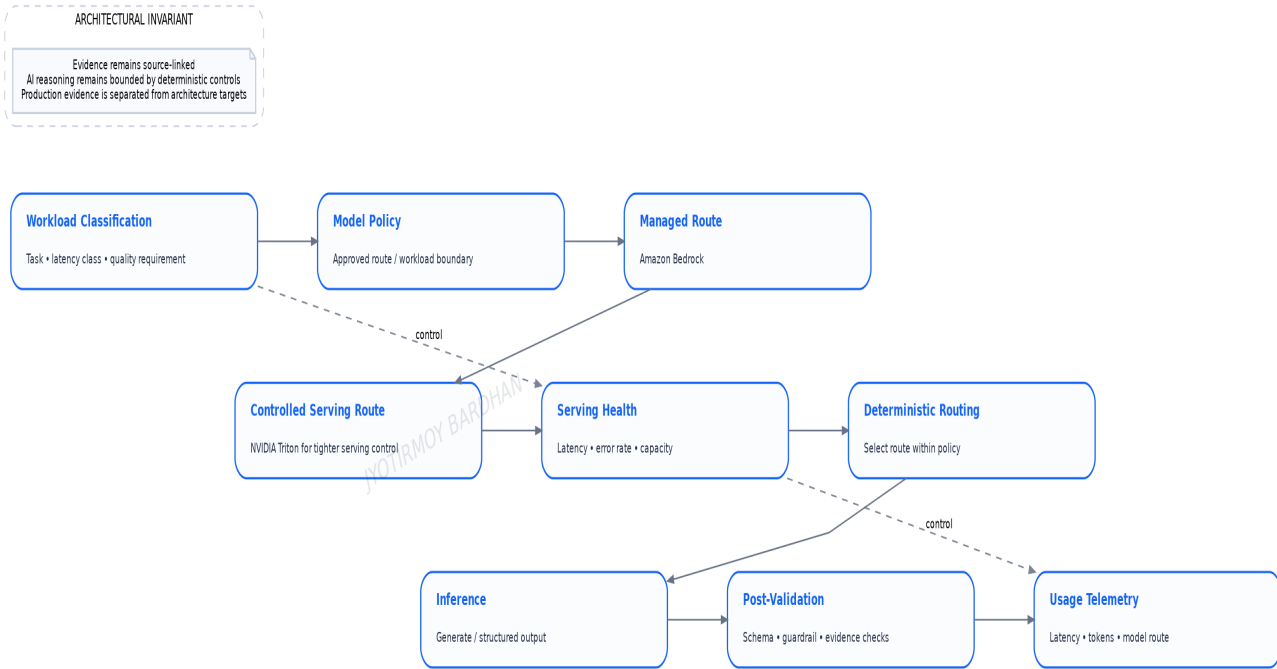
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Context engineering filters by authority, version and relevance, then bounds the payload before inference.

15 • MODEL GATEWAY & INFERENCE ROUTING

How does model gateway & inference routing behave as an explicit architecture boundary rather than a generic technology stack?

MODEL GATEWAY & INFERENCE ROUTING



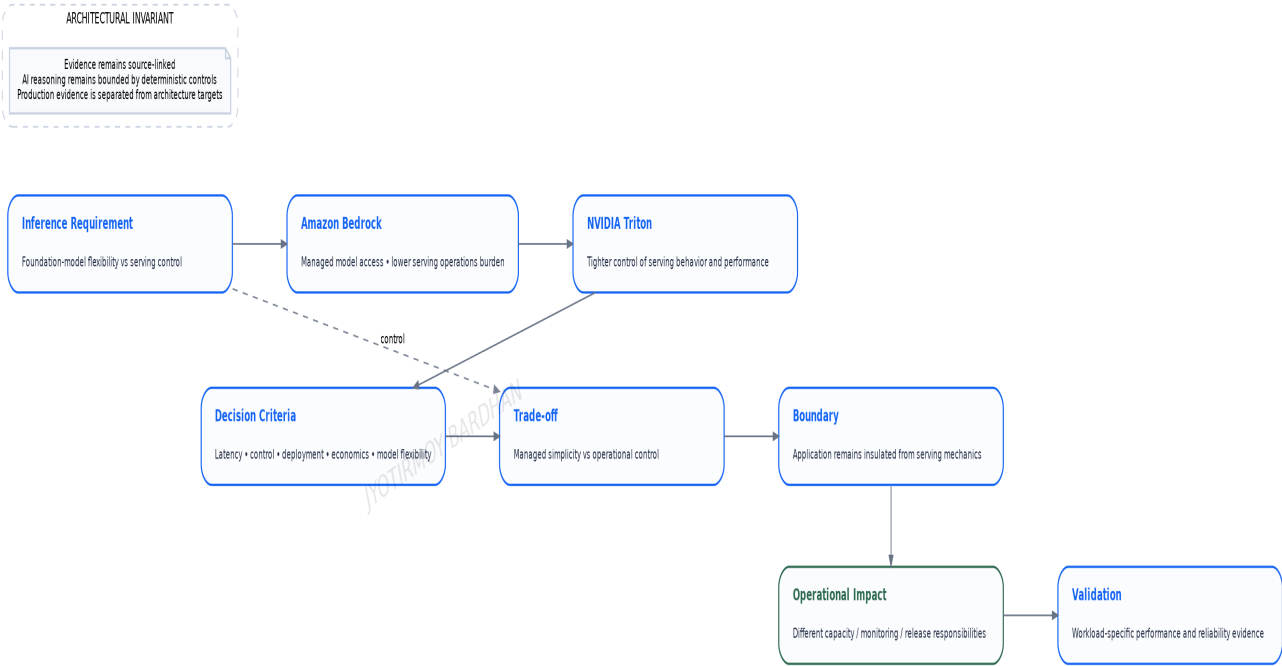
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Bedrock provides governed foundation-model access while Triton provides tighter serving control for selected workloads.

16 • MANAGED BEDROCK vs CONTROLLED TRITON DECISION

How does managed bedrock vs controlled triton decision behave as an explicit architecture boundary rather than a generic technology stack?

MANAGED BEDROCK vs CONTROLLED TRITON DECISION



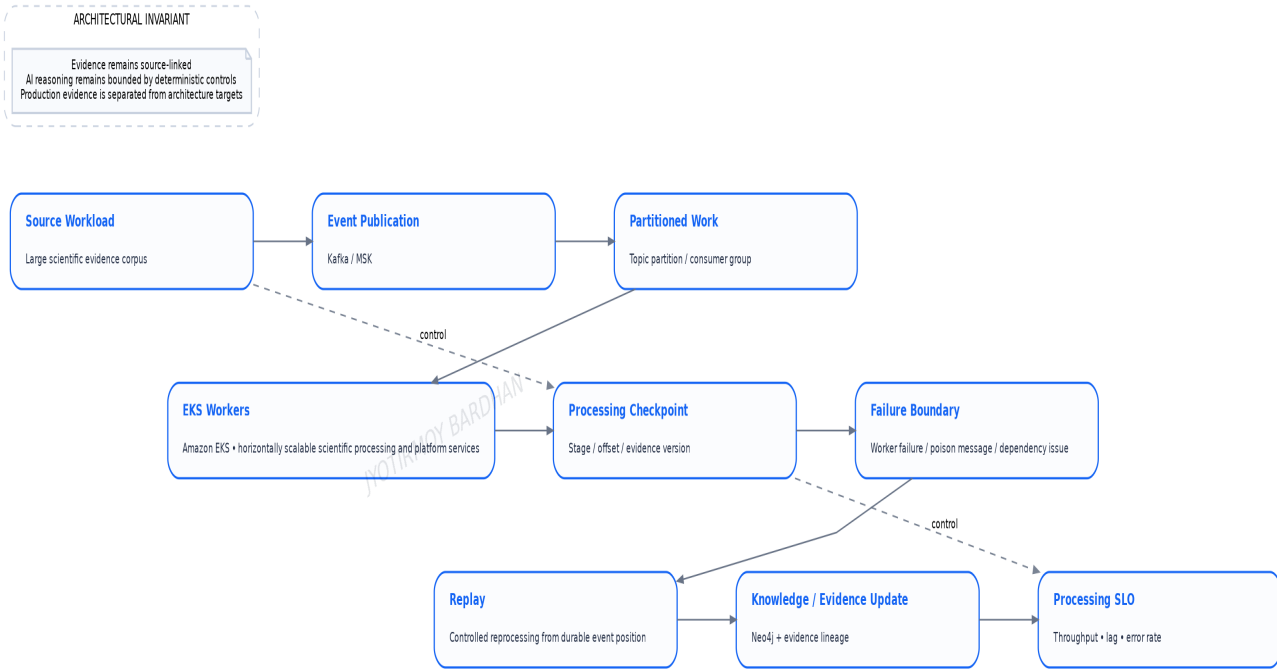
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

The managed-versus-controlled serving split balances operational burden, model flexibility, latency, performance control and economics.

17 • DISTRIBUTED SCIENTIFIC PROCESSING

How does distributed scientific processing behave as an explicit architecture boundary rather than a generic technology stack?

DISTRIBUTED SCIENTIFIC PROCESSING



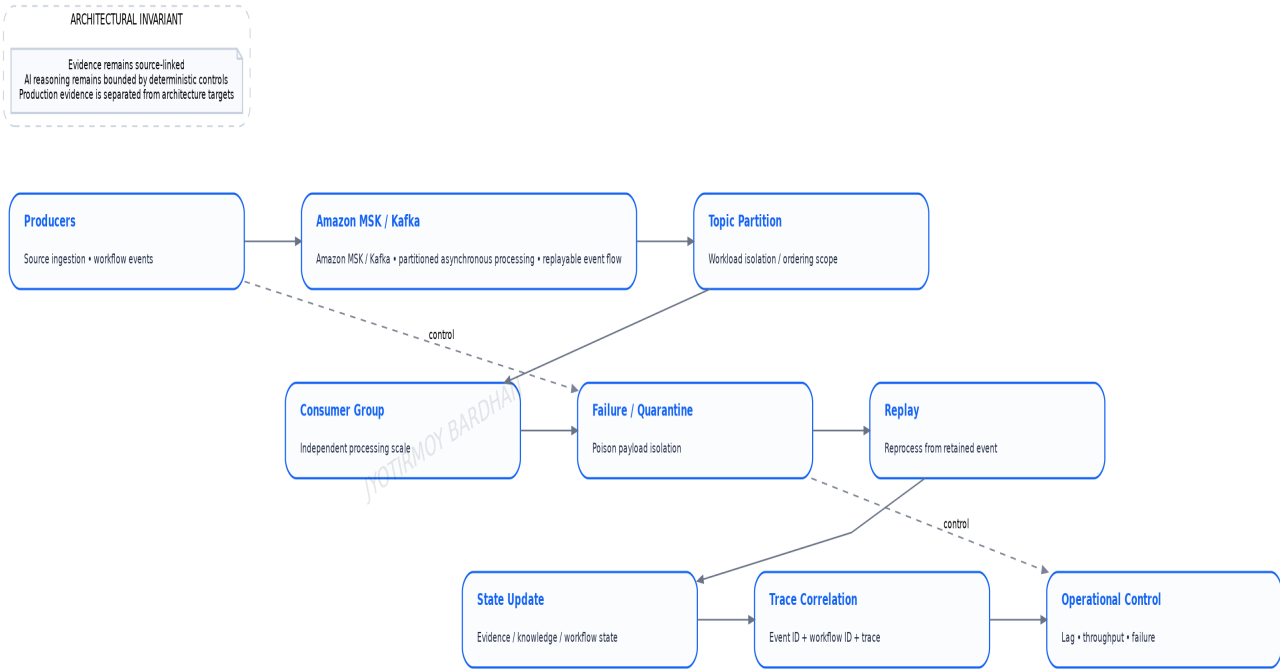
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

MSK/Kafka and EKS form the distributed processing substrate; checkpoint and replay semantics make processing recoverable.

18 • EVENT-DRIVEN ARCHITECTURE

How does event-driven architecture behave as an explicit architecture boundary rather than a generic technology stack?

EVENT-DRIVEN ARCHITECTURE



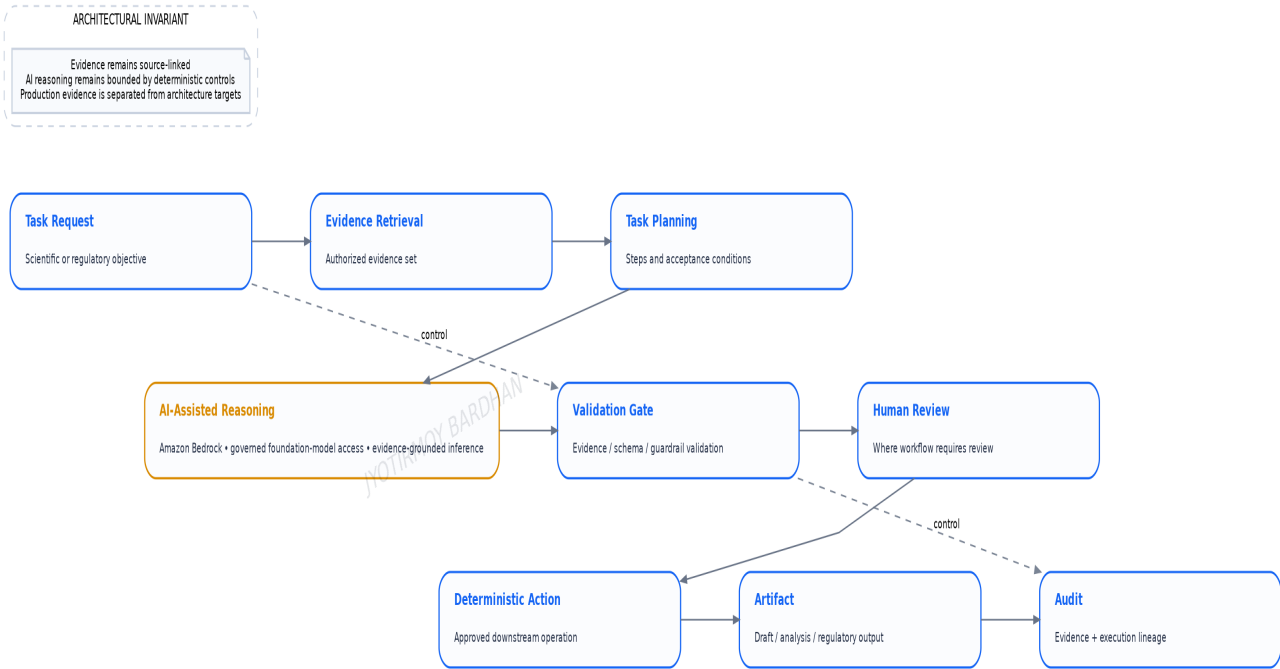
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Event-driven coordination decouples producers from consumers and makes lag, partitioning, replay and failure isolation explicit.

19 • SCIENTIFIC / REGULATORY WORKFLOW ORCHESTRATION

How does scientific / regulatory workflow orchestration behave as an explicit architecture boundary rather than a generic technology stack?

SCIENTIFIC / REGULATORY WORKFLOW ORCHESTRATION



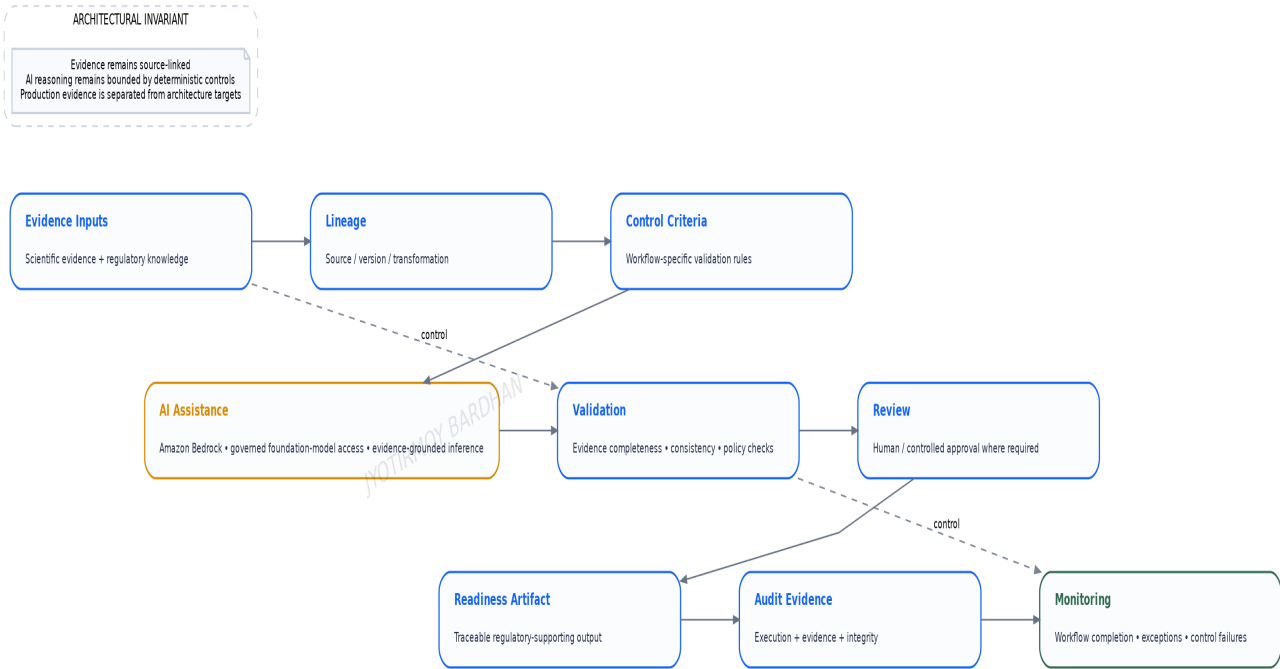
Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Scientific and regulatory workflows combine AI-assisted reasoning with deterministic validation, review and output controls.

20 • REGULATORY READINESS CONTROL PLANE

How does regulatory readiness control plane behave as an explicit architecture boundary rather than a generic technology stack?

REGULATORY READINESS CONTROL PLANE



Rendered architecture evidence • technical focus: boundaries, contracts, state, failure behaviour and production controls.

Regulatory readiness is represented as a control plane around evidence, lineage, validation and review rather than unconstrained generation.

16 • FLAGSHIP TECHNICAL CASE STUDY

Biomedical Intelligence, Validation & Regulatory Readiness Platform

The platform unified scientific literature, biomedical datasets, molecular interactions, clinical evidence, biomarker intelligence and regulatory knowledge into a common knowledge and reasoning architecture. The central problem was to create a controlled path from heterogeneous evidence to contextual retrieval, model inference and traceable output.

Architectural evidence	Implication
Platform profile	AWS • 24-member cross-functional engineering team • 20-month greenfield platform.
Architectural mandate	establish reusable boundaries across knowledge, retrieval, inference, cloud infrastructure, security, governance and production operations.
Production evidence	3×+ evidence-processing throughput; 40%+ manual research effort reduction; 30%+ regulatory workflow cycle reduction; sub-second scientific evidence retrieval; 95%+ provenance coverage; 70%+ repeatable workflow automation.

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

17 • ARCHITECTURE LIFECYCLE

Business intent → requirements → constraints → architecture → implementation → validation → production feedback

The working model starts with the architectural problem, not the technology. Scientific intent is decomposed into capability, evidence authority, NFRs, constraints, AI suitability, ownership boundaries and failure semantics before implementation choices are made.

Architectural evidence	Implication
Business intent → outcome contract.	Explicit boundary / production consequence
Requirement discovery → capability and interaction model.	Explicit boundary / production consequence
NFR analysis → measurable latency, throughput, provenance, security and reliability contract.	Explicit boundary / production consequence
Constraint analysis → eliminate designs that cannot satisfy scientific or governance boundaries.	Explicit boundary / production consequence
Architecture options → explicit trade-offs and ADRs.	Explicit boundary / production consequence
Validation → production evidence closes the loop.	Explicit boundary / production consequence

18 • AMBIGUITY → ARCHITECTURE

How ambiguous scientific requests become system contracts

Statements such as “unify biomedical intelligence” or “automate regulatory readiness” are insufficient engineering requirements. The architecture forces ambiguity into source authority, evidence, relationships, AI role, deterministic controls, latency, failure and reproducibility.

Architectural evidence	Implication
Separate business outcome from proposed technology.	Explicit boundary / production consequence
Identify authoritative versus derived information.	Explicit boundary / production consequence
Decide where graph relationships materially change retrieval behavior.	Explicit boundary / production consequence
Separate probabilistic reasoning from deterministic validation and workflow authority.	Explicit boundary / production consequence
Define evidence and workflow contracts before model integration.	Explicit boundary / production consequence

19 • REQUIREMENT DISCOVERY

Capability contracts and ownership

Every major capability receives an owning boundary, interaction contract, state semantics, failure semantics and acceptance evidence. This prevents multiple services from independently owning provenance, authorization, model selection or workflow state.

Architectural evidence	Implication
Scientific intelligence → retrieval / knowledge boundary.	Explicit boundary / production consequence
Evidence lineage → evidence boundary.	Explicit boundary / production consequence
AI reasoning → inference boundary.	Explicit boundary / production consequence
Repeatable regulatory workflow → deterministic workflow boundary.	Explicit boundary / production consequence
Production reliability → operations / SRE boundary.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

20 • FUNCTIONAL REQUIREMENTS

Capability-to-component mapping

The functional decomposition distinguishes scientific data processing, knowledge representation, evidence retrieval, AI-assisted reasoning, workflow automation, validation and audit evidence.

Architectural evidence	Implication
Evidence discovery → federated data + retrieval.	Explicit boundary / production consequence
Relationship-aware discovery → Neo4j / GraphRAG.	Explicit boundary / production consequence
Semantic matching → semantic retrieval.	Explicit boundary / production consequence
AI synthesis → Bedrock / controlled serving.	Explicit boundary / production consequence
Repeatable workflow → deterministic orchestration + validation.	Explicit boundary / production consequence
Traceability → provenance / audit chain.	Explicit boundary / production consequence

21 • NON-FUNCTIONAL REQUIREMENTS

Architecture-driving NFRs

NFRs directly shaped the architecture. The production evidence set demonstrates that the selected boundaries were not purely theoretical.

Architectural evidence	Implication
Performance	documented sub-second scientific evidence retrieval.
Throughput	3×+ scientific evidence-processing throughput.
Human efficiency	40%+ manual research effort reduction.
Workflow efficiency	30%+ regulatory workflow cycle reduction.
Traceability	95%+ evidence provenance coverage.
Automation	70%+ repeatable evidence/regulatory workflow automation.

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

22 • CONSTRAINTS & TRADE-OFF LEDGER

What could not simply be optimized away

Scientific workloads combine heterogeneous data, relationship-heavy knowledge, sensitive research information, variable inference workloads, provenance requirements and operational constraints. The architecture accepts complexity where it buys explicit control.

Architectural evidence	Implication
Federation adds integration complexity but preserves source ownership.	Explicit boundary / production consequence
GraphRAG adds graph lifecycle and query complexity but supports relationship-aware discovery.	Explicit boundary / production consequence
Kafka/MSK adds asynchronous operating complexity but provides decoupling and replay.	Explicit boundary / production consequence
Triton adds serving operations but enables tighter inference control for selected workloads.	Explicit boundary / production consequence
Provenance adds metadata and storage overhead but enables reconstruction.	Explicit boundary / production consequence

23 • ARCHITECTURAL BOUNDARY MODEL

Data • evidence • knowledge • context • inference • workflow • trust • operations

The platform is defined more by boundaries than by individual technologies. Each boundary has a purpose, an owner, a failure mode and an explicit prohibition against absorbing another boundary's authority.

Architectural evidence	Implication
Data boundary	source authority and access.
Evidence boundary	normalized evidence and lineage.
Knowledge boundary	graph and semantic representation.
Context boundary	admissibility, provenance and budget.
Inference boundary	model and serving abstraction.
Workflow boundary	deterministic state and validation.
Trust boundary	guardrails and security.
Operations boundary	telemetry and SLOs.

24 • GRAPHRAG ENGINEERING

Neo4j + semantic retrieval

GraphRAG is used where relationship structure materially contributes to scientific discovery. Semantic retrieval remains the candidate-generation mechanism for similarity-oriented queries; graph traversal supplies connected context and multi-hop evidence.

Architectural evidence	Implication
Entity extraction → canonical entities.	Explicit boundary / production consequence
Relationship extraction → scientific links.	Explicit boundary / production consequence
Graph update → Neo4j.	Explicit boundary / production consequence
Semantic index update → retrieval layer.	Explicit boundary / production consequence
Query → semantic candidates + graph expansion.	Explicit boundary / production consequence
Fusion → reranking → bounded evidence context.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

25 • EVIDENCE PROVENANCE

Source → transformation → evidence → knowledge → output

Provenance is treated as an architectural property rather than a reporting feature. The documented production outcome is 95%+ evidence provenance coverage.

Architectural evidence	Implication
Source identifier and version.	Explicit boundary / production consequence
Ingestion / transformation metadata.	Explicit boundary / production consequence
Evidence object identity.	Explicit boundary / production consequence
Knowledge / retrieval references.	Explicit boundary / production consequence
Context inclusion references.	Explicit boundary / production consequence
Generated artifact references.	Explicit boundary / production consequence
Audit / integrity metadata.	Explicit boundary / production consequence

26 • RETRIEVAL FAILURE LOCALIZATION

Separate retrieval failure from generation failure

The retrieval pipeline is intentionally decomposed so latency and correctness failures can be localized to ingestion, indexing, access filtering, candidate retrieval, graph traversal, reranking or context assembly rather than being blamed generically on the model.

Architectural evidence	Implication
Measure each retrieval stage independently.	Explicit boundary / production consequence
Keep source authority and access checks before model context.	Explicit boundary / production consequence
Record retrieval configuration versions.	Explicit boundary / production consequence
Treat graph and semantic retrieval as separate failure boundaries.	Explicit boundary / production consequence
Use production telemetry to identify the actual bottleneck.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

27 • CONTEXT ENGINEERING

Evidence budget and bounded model context

The model should receive a selected evidence package, not an uncontrolled document dump. Context selection considers authority, provenance, relevance, relationship structure, evidence diversity and payload size.

Architectural evidence	Implication
Authority → version → relevance → diversity → budget → context.	Explicit boundary / production consequence
Every selected item retains a source reference.	Explicit boundary / production consequence
Context construction is measured separately from retrieval.	Explicit boundary / production consequence
Context size is an inference-cost and latency control, not only a quality concern.	Explicit boundary / production consequence

28 • INFERENCE ARCHITECTURE

Amazon Bedrock + NVIDIA Triton

The inference boundary separates application workflow semantics from model-serving mechanics. Bedrock provides governed foundation-model access; Triton is reserved for workloads where tighter control of serving behavior, deployment topology or inference performance is required.

Architectural evidence	Implication
Workload classification selects the serving path.	Explicit boundary / production consequence
Provider/model policy remains deterministic.	Explicit boundary / production consequence
Serving health is measured independently.	Explicit boundary / production consequence
Post-inference validation remains outside model generation.	Explicit boundary / production consequence
Serving telemetry feeds SRE and FinOps decisions.	Explicit boundary / production consequence

29 • MODEL-SERVING TRADE-OFF

Managed versus controlled inference

The architecture deliberately avoids treating managed and self-managed serving as mutually exclusive. The decision is workload-specific and balances operational burden, model flexibility, latency, control and economics.

Architectural evidence	Implication
Bedrock	lower undifferentiated serving operations.
Triton	greater serving control for selected workloads.
Common application contract	workflow asks for inference capability, not provider-specific API semantics.
Validation	workload-specific performance and reliability evidence.

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

30 • DISTRIBUTED SCIENTIFIC PROCESSING

MSK/Kafka + EKS

Large evidence-processing workloads are decoupled through Kafka/MSK and executed by horizontally scalable EKS workers. This separates event production from processing capacity and makes replay semantics explicit.

Architectural evidence	Implication
Partition workload.	Explicit boundary / production consequence
Scale consumer groups independently.	Explicit boundary / production consequence
Track processing position and evidence version.	Explicit boundary / production consequence
Quarantine poison events.	Explicit boundary / production consequence
Replay from retained event position.	Explicit boundary / production consequence
Measure lag, throughput and failure rate.	Explicit boundary / production consequence

31 • WORKFLOW CONTROL

AI-assisted reasoning inside deterministic workflow

AI can propose synthesis, classification or next-step reasoning, while deterministic workflow and validation boundaries control what becomes an accepted artifact.

Architectural evidence	Implication
Request → evidence retrieval → AI reasoning → validation → review where required → artifact → audit.	Explicit boundary / production consequence
Validation is a separate stage, not a prompt instruction.	Explicit boundary / production consequence
High-impact workflow steps retain explicit approval / control semantics.	Explicit boundary / production consequence

32 • GUARDRAILS & TRUST

AI trust boundary

Bedrock Guardrails and evidence controls are placed around the model boundary. Retrieved material is not automatically treated as trusted instructions, and the model is not granted direct authority over deterministic operations.

Architectural evidence	Implication
Input / retrieval trust boundary.	Explicit boundary / production consequence
Pre-inference evidence and access checks.	Explicit boundary / production consequence
Model invocation.	Explicit boundary / production consequence
Guardrail evaluation.	Explicit boundary / production consequence
Output validation.	Explicit boundary / production consequence
Controlled downstream action.	Explicit boundary / production consequence
Audit evidence.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

33 • SECURITY RUNTIME CONTROLS

Zero-trust execution

Security is implemented at deterministic execution and data boundaries. Scientific evidence is accessed under explicit authorization, and sensitive information is not made trustworthy merely because a model can process it.

Architectural evidence	Implication
Identity and workload context.	Explicit boundary / production consequence
Least-privilege data access.	Explicit boundary / production consequence
Access-aware retrieval.	Explicit boundary / production consequence
Guardrails at inference.	Explicit boundary / production consequence
EKS / serving workload isolation.	Explicit boundary / production consequence
Security event telemetry.	Explicit boundary / production consequence
Audit evidence.	Explicit boundary / production consequence

34 • OBSERVABILITY TELEMETRY

OpenTelemetry + CloudWatch

Observability must correlate distributed scientific processing, retrieval and inference behavior. It should allow an operator to move from a production symptom to the responsible architecture boundary.

Architectural evidence	Implication
Tracing	request → processing → retrieval → inference → workflow.
Metrics	latency, throughput, lag, errors, saturation.
AI signals	route, inference latency, quality/evaluation signals.
Security	guardrail / access events.
SLI/SLO	production readiness and incident response.

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

35 • REAL-TIME MONITORING

Observe → classify → decide → act → verify

Real-time monitoring is treated as an operational control loop. The system should distinguish processing lag, retrieval latency, inference degradation, workflow failures and security anomalies so response is targeted.

Architectural evidence	Implication
Processing lag → scale / investigate consumers.	Explicit boundary / production consequence
Retrieval latency → isolate stage / tune retrieval.	Explicit boundary / production consequence
Inference degradation → route / constrain workload.	Explicit boundary / production consequence
Security anomaly → contain / investigate.	Explicit boundary / production consequence
Recovery → verify through telemetry, not merely traffic movement.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

36 • FAILURE MODE ENGINEERING

Failure containment across the pipeline

The architecture assumes dependency failure. The design objective is to prevent a local failure in ingestion, graph, retrieval, inference or workflow execution from becoming an uncontrolled platform-wide failure.

Architectural evidence	Implication
Source unavailable → isolate source / retry under bounded policy.	Explicit boundary / production consequence
Kafka poison event → quarantine.	Explicit boundary / production consequence
Worker loss → consumer rebalancing / replay semantics.	Explicit boundary / production consequence
Neo4j degradation → bounded retrieval failure path.	Explicit boundary / production consequence
Inference timeout → serving-specific recovery.	Explicit boundary / production consequence
Workflow failure → preserve execution evidence and state.	Explicit boundary / production consequence

37 • SCALABILITY ARCHITECTURE

Independent scaling domains

Scalability is designed component-by-component. Scientific processing, retrieval and inference have different bottlenecks and therefore should not be forced into one scaling policy.

Architectural evidence	Implication
Kafka	partition parallelism.
EKS processing	horizontal worker scaling.
Retrieval	independent service capacity.
Neo4j	graph workload / topology capacity planning.
Bedrock	managed service constraints and workload control.
Triton	dedicated serving capacity for selected workloads.

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

38 • DATA CONSISTENCY & REPRODUCIBILITY

Version-aware scientific AI

Reproducibility depends on preserving the versions of evidence, graph/index state, workflow logic and serving configuration that participated in an output.

Architectural evidence	Implication
Source version.	Explicit boundary / production consequence
Evidence transformation version.	Explicit boundary / production consequence
Graph / retrieval index state.	Explicit boundary / production consequence
Workflow version.	Explicit boundary / production consequence
Model / serving route metadata.	Explicit boundary / production consequence
Output artifact and evidence references.	Explicit boundary / production consequence

Inference as a production resource

Inference economics are connected to workload and context decisions. Cost optimization is subordinate to quality, safety and evidence requirements.

Architectural evidence	Implication
Classify workload.	Explicit boundary / production consequence
Control context size.	Explicit boundary / production consequence
Choose managed or controlled serving path.	Explicit boundary / production consequence
Measure usage.	Explicit boundary / production consequence
Attribute usage to task/workflow.	Explicit boundary / production consequence
Optimize without bypassing quality or safety controls.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

40 • EVALUATION QUALITY ENGINEERING

Evaluation as a release control

Evaluation is separated into retrieval, grounding, workflow, safety and performance dimensions. A single aggregate score would hide which architectural boundary actually regressed.

Architectural evidence	Implication
Retrieval	relevance / evidence selection.
Generation	grounding / task correctness.
Workflow	state and validation.
Safety	guardrail / adversarial behavior.
Performance	latency / throughput.
Release gate	accept / tune / reject.
Production failures become regression cases.	Explicit boundary / production consequence

41 • RED TEAMING

Adversarial architecture validation

Red teaming tests architecture boundaries rather than attempting to prove that a model is inherently safe. Confirmed failures become regression controls.

Architectural evidence	Implication
Direct prompt injection.	Explicit boundary / production consequence
Indirect injection through retrieved content.	Explicit boundary / production consequence
Data leakage / cross-scope access.	Explicit boundary / production consequence
Unsafe structured output.	Explicit boundary / production consequence
Resource exhaustion.	Explicit boundary / production consequence
RAG poisoning.	Explicit boundary / production consequence
Detection → containment → regression.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

42 • AUDITABILITY

Reconstructing an AI-assisted scientific decision

Auditability means reconstructing the actor, evidence, context, serving route, workflow, validation and resulting artifact. Operational logs alone are not treated as sufficient evidence.

Architectural evidence	Implication
Identity / execution context.	Explicit boundary / production consequence
Evidence references.	Explicit boundary / production consequence
Context package.	Explicit boundary / production consequence
Model / serving metadata.	Explicit boundary / production consequence
Guardrail / validation outcome.	Explicit boundary / production consequence
Workflow state.	Explicit boundary / production consequence
Artifact.	Explicit boundary / production consequence
Integrity metadata.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

43 • CLOUD PLATFORM ENGINEERING

AWS production substrate

Cloud infrastructure is the deterministic substrate. EKS hosts scalable platform workloads while AWS data, messaging and inference services form explicit integration boundaries.

Architectural evidence	Implication
EKS	platform and processing runtime.
MSK/Kafka	asynchronous processing.
AWS data services	governed scientific data.
Neo4j	knowledge layer.
Bedrock	managed inference.
Triton	controlled serving.
Terraform / GitHub Actions	reproducible delivery.
OTel / CloudWatch	operations.

44 • INFRASTRUCTURE AS CODE

Terraform + GitHub Actions

Infrastructure and deployment behavior are versioned so architecture changes can be reviewed, reproduced and rolled forward or back under controlled release practices.

Architectural evidence	Implication
Version infrastructure topology.	Explicit boundary / production consequence
Validate changes before deployment.	Explicit boundary / production consequence
Promote through controlled environments.	Explicit boundary / production consequence
Correlate deployment version with evaluation evidence.	Explicit boundary / production consequence
Keep production configuration distinguishable from runtime state.	Explicit boundary / production consequence

45 • PRODUCTION READINESS

Evidence before go-live

Production readiness is evidence-driven. The acceptance package must cover architecture, security, AI quality, data provenance, reliability, observability and operational behavior.

Architectural evidence	Implication
Architecture review complete.	Explicit boundary / production consequence
Security / guardrail controls validated.	Explicit boundary / production consequence
Retrieval and grounding evaluation passed.	Explicit boundary / production consequence
Failure and recovery paths exercised.	Explicit boundary / production consequence
Telemetry and SLOs active.	Explicit boundary / production consequence
Deployment reproducible.	Explicit boundary / production consequence
Audit / provenance evidence available.	Explicit boundary / production consequence

46 • INCIDENT INVESTIGATION

Causal path from symptom to boundary

A useful incident trace follows source/event → processing → evidence → retrieval → context → inference → workflow → output. This avoids attributing every production problem to the model.

Architectural evidence	Implication
Latency spike → locate retrieval / inference / processing stage.	Explicit boundary / production consequence
Quality regression → correlate evaluation configuration.	Explicit boundary / production consequence
Processing backlog → inspect Kafka lag / worker capacity.	Explicit boundary / production consequence
Workflow failure → inspect state and validation boundary.	Explicit boundary / production consequence
Security anomaly → correlate access and guardrail events.	Explicit boundary / production consequence

47 • TRACEABILITY

Requirement → ADR → mechanism → evidence → outcome

Architecture documentation remains credible when every major claim can be traced to the problem it solved, the constraint that shaped the decision, the mechanism that implemented it and the production evidence that followed.

Architectural evidence	Implication
Requirement.	Explicit boundary / production consequence
Constraint.	Explicit boundary / production consequence
Rejected alternative.	Explicit boundary / production consequence
ADR decision.	Explicit boundary / production consequence
Implementation mechanism.	Explicit boundary / production consequence
Validation evidence.	Explicit boundary / production consequence
Production outcome.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

48 • ARCHITECTURE DECISION RECORDS

Core ADR set

The portfolio uses ADRs to document decisions such as federated data architecture, GraphRAG, distributed processing, managed versus controlled inference, provenance, guardrails, observability and IaC.

Architectural evidence	Implication
ADR-01 Federated scientific data.	Explicit boundary / production consequence
ADR-02 GraphRAG + semantic retrieval.	Explicit boundary / production consequence
ADR-03 MSK/Kafka + EKS.	Explicit boundary / production consequence
ADR-04 Bedrock + Triton serving boundary.	Explicit boundary / production consequence
ADR-05 Evidence provenance.	Explicit boundary / production consequence
ADR-06 Guardrails / trust boundary.	Explicit boundary / production consequence
ADR-07 OTel / CloudWatch.	Explicit boundary / production consequence
ADR-08 Terraform / GitHub Actions.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

49 • PROJECT LEADERSHIP & AUTHORITY

Principal AI Architect contribution

The architectural contribution was target-state definition, platform decomposition, boundary ownership and cross-functional technical authority across knowledge, retrieval, inference, cloud infrastructure, security, governance and production operations.

Architectural evidence	Implication
Defined target-state scientific AI architecture.	Explicit boundary / production consequence
Established platform-versus-domain boundaries.	Explicit boundary / production consequence
Drove managed-versus-controlled inference trade-off.	Explicit boundary / production consequence
Established reusable patterns for governed scientific AI workloads.	Explicit boundary / production consequence
Connected architecture decisions to production evidence.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

50 • PRODUCTION OUTCOME MAP

Architecture decision → measurable consequence

The production outcomes are presented as evidence of architectural effect, not as generic marketing claims.

Architectural evidence	Implication
Evidence-processing throughput → 3×+.	Explicit boundary / production consequence
Manual research effort → 40%+ reduction.	Explicit boundary / production consequence
Regulatory workflow cycle → 30%+ reduction.	Explicit boundary / production consequence
Evidence retrieval → sub-second.	Explicit boundary / production consequence
Provenance coverage → 95%+.	Explicit boundary / production consequence
Repeatable workflow automation → 70%+.	Explicit boundary / production consequence

51 • WHAT I WOULD DEFEND IN A CTO REVIEW

Architecture questions that matter

The defensible narrative is not “we used AI.” It is why the platform separated evidence, knowledge, inference and workflow; why GraphRAG was needed; why asynchronous processing was accepted; and why managed and controlled inference coexist.

Architectural evidence	Implication
What is authoritative?	Explicit boundary / production consequence
What can the model decide?	Explicit boundary / production consequence
What must remain deterministic?	Explicit boundary / production consequence
What happens when retrieval or inference fails?	Explicit boundary / production consequence
How is an output reconstructed?	Explicit boundary / production consequence
How do production metrics prove the architecture works?	Explicit boundary / production consequence

52 • WHAT I DELIBERATELY DID NOT ABSTRACT

Avoiding architecture overreach

Not every domain behavior belongs in a platform abstraction. The architecture centralizes reusable infrastructure and trust boundaries while leaving scientific/regulatory semantics to domain-owned workflow components.

Architectural evidence	Implication
Platform owns	evidence access, retrieval infrastructure, inference boundary, telemetry, security patterns.
Domain owns	scientific task semantics, validation criteria, workflow-specific business rules.
Model does not own	source authority, authorization, audit truth or deterministic side effects.

53 • FAILURE-DRIVEN ARCHITECTURE

Designing from what can go wrong

The architecture is evaluated by failure modes: source degradation, event duplication, worker loss, graph unavailability, retrieval timeout, model failure, malformed output and workflow validation failure.

Architectural evidence	Implication
Localize failures.	Explicit boundary / production consequence
Bound retries.	Explicit boundary / production consequence
Preserve evidence.	Explicit boundary / production consequence
Separate model failure from workflow side effects.	Explicit boundary / production consequence
Make recovery observable.	Explicit boundary / production consequence
Turn production failures into regression tests.	Explicit boundary / production consequence

54 • SECURITY + OBSERVABILITY CO-DESIGN

Security events are operational signals

Security monitoring is integrated into the same operational fabric as platform monitoring. Access denial, guardrail events and anomalous AI behavior must be correlated to the workload and trace that produced them.

Architectural evidence	Implication
Identity → access decision → retrieval → model → guardrail → output.	Explicit boundary / production consequence
Security events carry correlation context.	Explicit boundary / production consequence
Operational response can isolate a workload without losing audit evidence.	Explicit boundary / production consequence

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

55 • RESPONSIBLE AI SYSTEM PROPERTY

Grounding, provenance and human authority

Responsible AI is implemented as a system property. Evidence provenance, controlled context, validation, guardrails and explicit human review points provide concrete mechanisms for trustworthy AI-assisted workflows.

Architectural evidence	Implication
Evidence grounding.	Explicit boundary / production consequence
Source provenance.	Explicit boundary / production consequence
Controlled context.	Explicit boundary / production consequence
Guardrail enforcement.	Explicit boundary / production consequence
Human review where required.	Explicit boundary / production consequence
Decision / output traceability.	Explicit boundary / production consequence

56 • REAL-TIME CONTROL LOOP

Production telemetry → architectural action

The real-time control loop connects operational signals to targeted response. It avoids a single generic “AI health” indicator that cannot localize the cause.

Architectural evidence	Implication
Processing lag → worker capacity.	Explicit boundary / production consequence
Retrieval latency → retrieval boundary.	Explicit boundary / production consequence
Inference error → serving boundary.	Explicit boundary / production consequence
Guardrail anomaly → trust boundary.	Explicit boundary / production consequence
Quality regression → release / evaluation boundary.	Explicit boundary / production consequence

57 • SCALABILITY TARGET VS PRODUCTION EVIDENCE

Do not confuse capability with measured scale

The portfolio distinguishes architectural scalability from validated production evidence. The documented evidence supports independent scaling and specific measured outcomes; unsupported concurrency figures are not introduced.

Architectural evidence	Implication
Architecture capability	independent processing / retrieval / serving scaling.
Measured evidence	3×+ evidence-processing throughput.
Measured evidence	sub-second retrieval.
Measured evidence	70%+ repeatable workflow automation.

Principal AI Architect lens: start with the constraint, introduce the boundary, state the rejected alternative, explain runtime semantics, and close with production evidence.

58 • PRODUCTION OPERATING MODEL

Run the architecture after go-live

Production operation requires ownership of SLOs, incidents, evaluation, security, evidence lineage, deployment and cost—not only application support.

Architectural evidence	Implication
SRE owns service health.	Explicit boundary / production consequence
AI/platform owners own model and retrieval behavior.	Explicit boundary / production consequence
Data owners own source authority.	Explicit boundary / production consequence
Security/governance owns control posture.	Explicit boundary / production consequence
Architecture owns cross-boundary evolution.	Explicit boundary / production consequence